Angular

Web Development — Lecture 4

Seitbekov Sanzhar

KAZAKH-BRITISH TECHNICAL UNIVERSITY

KBTU · 2025 / 2026

Lecture Plan

PART 1 · 50 MIN

Introduction to Angular

→ What is Angular & why use it

→ Angular history: from AngularJS to v21

→ Architecture: components, templates, services

→ JavaScript vs TypeScript

→ TypeScript essentials for Angular

PART 2 · 50 MIN

Angular CLI & First App

→ Installing Node.js 24 & Angular CLI

→ Creating a project with ng new

→ Project structure walkthrough

→ Components, templates & data binding

→ Live coding: building our first app

What is Angular?

FRAMEWORK

Full platform — routing, forms, HTTP, testing out of the box

TYPESCRIPT

Built with and for TypeScript — type safety by design

GOOGLE

Maintained by Google, used in Gmail, Google Cloud Console, Firebase

ENTERPRISE

Opinionated structure — ideal for large teams & long-lived projects

Framework vs Library: React is a library — you pick your own router, state manager, form handler. Angular is a full framework — batteries included. Think of it as the difference between buying parts vs. a complete car.

2025: Angular v21 released November 2025. Zoneless by default, Signal Forms, Vitest, standalone components. Two major releases per year.

Angular Evolution

From AngularJS to Angular 21 — a complete rewrite and continuous improvement

2010
AngularJS (v1)
Two-way binding
JavaScript

2016
Angular 2
Complete rewrite
TypeScript

2020
Angular 9-11
Ivy renderer
Smaller bundles

2023
Angular 16-17
Signals introduced
Standalone default

2025
Angular 21
Zoneless default
Signal Forms

Key point: AngularJS (v1) and Angular (v2+) are completely different frameworks. Angular 2+ was a ground-up rewrite in TypeScript. When we say "Angular" today, we mean Angular 2+.

Angular Architecture

Components

Building blocks of UI. Each component has a TypeScript class, an HTML template, and CSS styles. Organized in a tree.

Templates

HTML with Angular syntax: @if, @for, {{ interpolation }}, [property] and (event) binding.

Services

Business logic & data fetching. Injected into components via Dependency Injection. Singleton by default.

Router

Maps URLs to components. Supports lazy loading, guards, and nested routes.

Signals

Reactive primitives for state management. signal(), computed(), effect(). The future of Angular reactivity.

Anatomy of a Component

```ts
import { Component, signal } from
'@angular/core';

@Component({
  selector: 'app-hello',
  template: `
    <h1>Hello, {{ name() }}!</h1>
    <button (click)="changeName()">
      Change Name
    </button>
  `,
  styles: `h1 { color: #dd0031; }`
})
export class HelloComponent {
  name = signal('World');

  changeName() {
    this.name.set('Angular 21');
  }
}
```

Key parts

→ @Component — decorator that marks a class as a component

→ selector — custom HTML tag name: <app-hello>

→ template — the HTML view with Angular syntax

→ styles — scoped CSS (won't leak to other components)

→ signal() — reactive state that auto-updates the view

→ (click) — event binding to a method

Standalone by default: In Angular 21, all components are standalone — no NgModules needed.

02

JavaScript vs TypeScript

Why Angular chose TypeScript — and why you should too

JavaScript vs TypeScript

JavaScript

```js
function greet(name) {
  return "Hello, " + name;
}

greet(42);  // "Hello, 42" — no error!
greet();     // "Hello, undefined" — silent bug

const user = { name: "Ali" };
console.log(user.age); // undefined, no warning
```

TypeScript

```ts
function greet(name: string): string {
  return "Hello, " + name;
}

greet(42);  // ✗ Error: number is not string
greet();     // ✗ Error: expected 1 argument

const user: { name: string } = { name: "Ali" };
console.log(user.age); // ✗ Error: 'age' does not exist
```

TypeScript = JavaScript + Types. Every valid JS file is valid TS. TypeScript adds type annotations that catch bugs at compile time, not in production. It compiles down to plain JavaScript.

TypeScript Essentials: Types

Basic types

```ts
// Primitives
let name: string = "Aidar";
let age: number = 20;
let active: boolean = true;

// Arrays
let scores: number[] = [95, 88, 72];
let names: string[] = ["Ali", "Sara"];

// Union types
let id: string | number = "abc";
id = 123; // also valid
```

Interfaces

```ts
interface Student {
  name: string;
  gpa: number;
  courses: string[];
  graduated?: boolean; // optional
}

const student: Student = {
  name: "Daulet",
  gpa: 3.8,
  courses: ["Web Dev", "Algo"]
};

// Error if you forget required fields
// Error if you add unknown fields
```

TypeScript for Angular

Classes with access modifiers

```ts
class UserService {
  private users: string[] = [];

  addUser(name: string): void {
    this.users.push(name);
  }

  getUsers(): string[] {
    return [...this.users];
  }
}

const svc = new UserService();
svc.addUser("Ali");
svc.users; // ❌ Error: 'users' is private
```

Generics & enums

```ts
// Generics — reusable typed containers
function first<T>(arr: T[]): T {
  return arr[0];
}

first<number>([1, 2, 3]);    // 1
first<string>(["a", "b"]); // "a"

// Enums — named constants
enum Role {
  Student = "STUDENT",
  Lecturer = "LECTURER",
  Admin = "ADMIN"
}

const myRole: Role = Role.Student;
```

Why it matters: Angular uses classes for components and services, decorators (@Component, @Injectable) to configure them, and interfaces to define data shapes. TypeScript makes all of this type-safe.

JS vs TS — Quick Comparison

Feature	JavaScript	TypeScript
Typing	Dynamic (runtime)	Static (compile-time)
Error detection	In browser / production	In your IDE before you run
Interfaces	❌ Not available	✅ interface, type
Access modifiers	❌ No private/public	✅ private, protected, public
Decorators	❌ Stage 3 proposal	✅ Used everywhere in Angular
Generics	❌	✅ <T>
IDE support	Good	Excellent — autocomplete, refactoring
Runs in browser?	Yes, directly	Compiled to JS first

> **Bottom line:** TypeScript is #1 on GitHub (Aug 2025). Angular, Vue, Svelte, Deno, Bun — all built with TypeScript. Learning TS is not optional in 2025.

03

Angular CLI & First App

From zero to running application in 3 minutes

Setting Up the Environment

Step 1: Install Node.js 24

```bash
# Download from nodejs.org or use nvm
node --version   # v24.x.x
npm --version    # 11.x.x
```

Step 2: Install Angular CLI

```bash
npm install -g @angular/cli

ng version
# Angular CLI: 21.x.x
# Node: 24.x.x
# Package Manager: npm 11.x.x
```

Step 3: Create a project

```bash
ng new my-first-app

# CLI asks:
# ? Which stylesheet format? → CSS
# ? Enable SSR? → No

cd my-first-app
ng serve
```

> **Open http://localhost:4200** — your Angular app is running! 🚀 The dev server watches for file changes and auto-reloads.

Project Structure

TREE

```
my-first-app/
├── src/
│   ├── app/
│   │   ├── app.ts          ← root component
│   │   └── app.routes.ts   ← routing config
│   ├── main.ts             ← bootstrap entry
│   ├── index.html          ← single HTML page
│   └── styles.css          ← global styles
├── angular.json            ← CLI configuration
├── package.json            ← npm dependencies
└── tsconfig.json           ← TypeScript config
```

Key files

→ main.ts — bootstraps the app with `bootstrapApplication()`

→ app.ts — root component (v21 uses 2025 naming convention)

→ app.routes.ts — defines URL → component mapping

→ angular.json — build settings, dev server config, assets

→ index.html — the single page; contains `<app-root>`

SPA: Angular is a Single Page Application framework. One index.html, JavaScript handles all navigation.

How the App Starts

main.ts

```ts
import { bootstrapApplication } from
  '@angular/platform-browser';
import { AppComponent } from './app/app';
import { appConfig } from './app/app.config';

bootstrapApplication(
  AppComponent,
  appConfig
);
```

index.html

```html
<!DOCTYPE html>
<html>
<head>
  <title>MyFirstApp</title>
</head>
<body>
  <!-- Angular replaces this -->
  <app-root></app-root>
</body>
</html>
```

Flow: Browser loads index.html → main.ts runs → Angular bootstraps AppComponent → `<app-root>` gets replaced with your component's template.

Data Binding in Angular

```ts
@Component({
  selector: 'app-root',
  template: `
    <!-- Interpolation -->
    <h1>{{ title }}</h1>

    <!-- Property binding -->
    <img [src]="imageUrl">

    <!-- Event binding -->
    <button (click)="onClick()">
      Click me
    </button>

    <!-- Two-way binding (forms) -->
    <input [(ngModel)]="name">
    <p>Hello, {{ name }}!</p>
  `
})
export class AppComponent {
  title = 'My App';
  imageUrl = 'logo.png';
  name = '';

  onClick() {
```

4 types of binding

→ {{ value }} — Interpolation: display data in HTML

→ [property]="expr" — Property binding: set DOM property

→ (event)="handler()" — Event binding: react to events

→ [(ngModel)]="var" — Two-way: sync input ↔ variable

Remember the syntax

[square] = data goes **IN** to the element

(round) = data comes **OUT** of the element

[(banana in box)] = **BOTH** directions

Template Control Flow

@if / @else

```html
<!-- Conditional rendering -->
@if (isLoggedIn) {
  <p>Welcome back, {{ username }}!</p>
} @else {
  <p>Please log in.</p>
}
```

@for

```html
<!-- Loop through a list -->
<ul>
  @for (student of students; track student.id) {
    <li>{{ student.name }} — {{ student.gpa }}
</li>
  } @empty {
    <li>No students found.</li>
  }
</ul>
```

Built-in control flow (Angular 17+)

→ @if / @else if / @else — replaces *ngIf

→ @for with track — replaces *ngFor, must have track

→ @switch / @case — replaces ngSwitch

→ @empty — fallback when @for list is empty

Modern Angular: The new control flow syntax (with @) is the standard since Angular 17. No imports needed — it's built into the template compiler. Faster and more readable than the old *ngIf/*ngFor directives.

Signals — Reactive State

```ts
import { Component, signal, computed } from
  '@angular/core';

@Component({
  selector: 'app-counter',
  template: `
    <h2>Count: {{ count() }}</h2>
    <p>Double: {{ double() }}</p>
    <button (click)="increment()">+1</button>
    <button (click)="reset()">Reset</button>
  `
})
export class CounterComponent {
  // Writable signal
  count = signal(0);

  // Computed signal (auto-updates)
  double = computed(() => this.count() * 2);

  increment() { this.count.update(n => n + 1); }
  reset() { this.count.set(0); }
}
```

Signal API

→ signal(value) — create reactive state

→ count() — read the current value (call it!)

→ .set(newVal) — replace the value

→ .update(fn) — update based on previous value

→ computed(() => ...) — derived value, auto-updates

→ effect(() => ...) — side effects on signal change

Angular 21: Signals are the core of Angular's new zoneless architecture. No more Zone.js — only the components that read a changed signal re-render.

Angular CLI Commands

Essential commands

BASH

```bash
# Create new project
ng new my-app

# Start dev server
ng serve

# Generate a component
ng generate component header
# shorthand: ng g c header

# Generate a service
ng generate service api
# shorthand: ng g s api

# Build for production
ng build

# Run tests
ng test
```

What ng g c header creates

TREE

```tree
src/app/header/
├── header.ts        ← component class
├── header.html      ← template
├── header.css       ← scoped styles
└── header.spec.ts   ← unit test
```

CLI does the boring work

→ Creates files with correct naming conventions

→ Sets up imports and decorators

→ Generates a ready-to-use test file

→ All components are standalone by default

Combining everything: components + signals + @for + event binding

Interface

```ts
import { Component, signal } from
'@angular/core';

interface Student {
  id: number;
  name: string;
  gpa: number;
}
```

Template

```html
<h1>Student List</h1>
<input #nameInput placeholder="Name">
<input #gpaInput type="number"
placeholder="GPA">
<button (click)="add(nameInput, gpaInput)">
  Add Student
</button>

<ul>
  @for (s of students(); track s.id) {
    <li [class.honors]="s.gpa >= 3.5">
      {{ s.name }} — GPA: {{ s.gpa }}
      <button (click)="remove(s.id)">×</button>
    </li>
  } @empty {
    <li>No students yet.</li>
  }
```

Template concepts

→ #nameInput — template reference variable

→ [class.honors] — conditional CSS class

→ (click)="add()" — event binding

→ @for ... track s.id — loop with tracking

→ @empty — fallback when list is empty

→ {{ s.name }} — interpolation

🎯 Student List App — Class Logic

```ts
export class AppComponent {
  students = signal<Student[]>([]);
  nextId = 1;

  add(nameEl: HTMLInputElement,
      gpaEl: HTMLInputElement) {
    const name = nameEl.value.trim();
    const gpa = parseFloat(gpaEl.value);
    if (!name || isNaN(gpa)) return;

    this.students.update(list =>
      [...list, {
        id: this.nextId++,
        name,
        gpa
      }]
    );

    nameEl.value = '';
    gpaEl.value = '';
  }

  remove(id: number) {
    this.students.update(list =>
      list.filter(s => s.id !== id)
    );
  }
}
```

Class concepts

→ signal<Student[]> — typed reactive state

→ .update(fn) — immutable update via callback

→ [...list, newItem] — spread = new array (immutable)

→ .filter() — remove item without mutating

→ parseFloat + isNaN — input validation

→ nameEl.value = '' — clear inputs after add

Key pattern: Never mutate signal values directly. Always use .update() with a callback that returns a **new** array/object. Angular detects changes by reference, not deep comparison.

ANGULAR 21

What's New in Angular 21 (Nov 2025)

⚡

Zoneless Default

No more Zone.js. Signals drive change detection. Smaller bundles, faster apps, cleaner stack traces.

📝

Signal Forms

Experimental new forms API built on Signals. No more valueChanges subscriptions. Automatic sync.

🧪

Vitest Default

Goodbye Karma. Vitest is the new default test runner — faster, modern mocking, snapshot testing.

♿

Angular Aria

New headless accessible components. 8 UI patterns, 13 components. You bring the CSS, Angular handles ARIA.

🎨

Tailwind + 2025 Style Guide

Bundled Tailwind setup on `ng new`. New concise file naming: `app.ts` instead of `app.component.ts`.

Lecture Summary

What we covered

→ Angular — full framework by Google, TypeScript-first

→ Components = class + template + styles

→ TypeScript: types, interfaces, classes, generics

→ Angular CLI: ng new, ng serve, ng generate

→ Data binding: {{ }}, [prop], (event), [(ngModel)]

→ Control flow: @if, @for, @switch

→ Signals: signal(), computed(), .set(), .update()

For practice

→ Install Node.js 24 & Angular CLI 21

→ Create a new project with ng new

→ Build a Student List app with add/delete

→ Use signals, @for with track, event binding

→ Bonus: add sorting, filtering, GPA average

Resources

→ Angular Docs — angular.dev

→ TypeScript Handbook — typescriptlang.org/docs

→ Angular Tutorial — angular.dev/tutorials

?

Q & A

Ask anything — we'll debug it together.

How to reach me

Seitbekov Sanzhar

Lecturer

s.seitbekov@kbtu.kz

Office 178

Course chat

Microsoft Teams

Team code: **ksg13tw**

→ Announcements + materials

→ Homework questions

→ Useful links (cheatsheets, docs)